

Conceitos Básicos

- **Algoritmo:**
- **Estrutura de Dados:**
- **Programa:**

Conceitos Básicos

- **Algoritmo:**

- Um algoritmo é qualquer procedimento computacional bem definido que toma algum valor ou conjunto de valores como entrada e produz algum valor ou conjunto de valores como saída.

- **Estrutura de Dados:**

- Uma estrutura de dados é uma forma organizada de armazenar, gerenciar e organizar dados em um computador para que eles possam ser utilizados de maneira eficiente

- **Programa:**

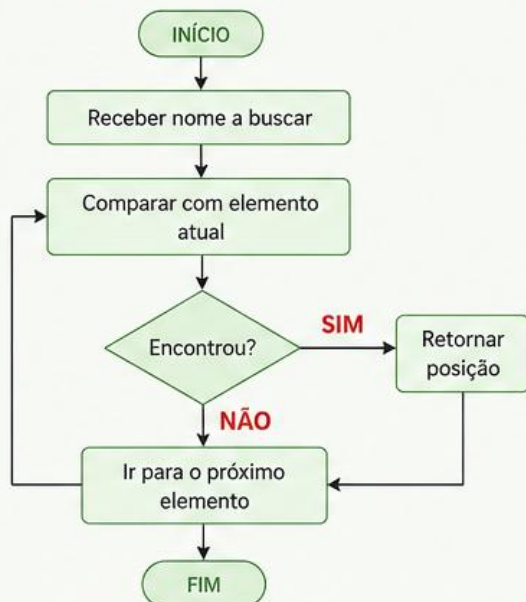
- Um programa é a tradução prática, concreta e executável de um algoritmo em uma linguagem de programação específica

Lógica

ALGORITMO

Define o passo a passo lógico para resolver um problema.

Exemplo: Busca de um nome



O algoritmo define
O QUE fazer.

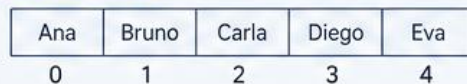
Organização

ESTRUTURA DE DADOS

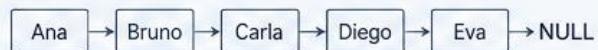
Define como os dados são organizados e armazenados para serem usados com eficiência.

Exemplos de estruturas:

- VETOR (ARRAY)



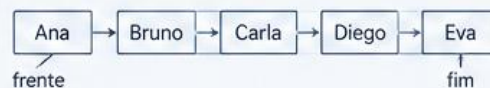
- LISTA LIGADA



- PILHA (STACK)



- FILA (QUEUE)



A estrutura de dados define
COMO os dados serão organizados
para que o algoritmo seja eficiente.

Execução

PROGRAMA

Implementa o algoritmo usando uma ou mais estruturas de dados em uma linguagem de programação.

Exemplo (Python):

```
def busca_linear(lista, nome):  
    for i in range(len(lista)):  
        if lista[i] == nome:  
            return i  
    return -1  
  
# Exemplo de uso  
nomes = ["Ana", "Bruno", "Carla", "Diego", "Eva"]  
busca = "Carla"  
pos = busca_linear(nomes, busca)  
if pos != -1:  
    print(f"Encontrado na posição {pos}")  
else:  
    print("Não encontrado")
```

O programa executa,
combinando algoritmo e
estrutura de dados.

RESULTADO

A combinação correta de **ALGORITMO** + **ESTRUTURA DE DADOS**
no **PROGRAMA** gera soluções corretas e eficientes!



Estrutura de Dados

- Uma **Estrutura de Dados** pode ser dividida em dois pilares fundamentais: **dado** e **estrutura**

Dado

Elemento que possui valor agregado e que pode ser utilizado para solucionar problemas computacionais. Os dados possuem tipos específicos.

Estrutura

Elemento estrutural que responsável por carregar as informações dentro de uma estrutura de *software*.

Dado

Tipos de dados:

- Inteiro (int)
- Texto (string)
- Caracter (char)
- Ponto flutuante (float)
- Ponto flutuante (double)

Estrutura

Estruturas:

- Vetores multidimensionais
- Pilhas
- Filas
- Listas

TIPOS DE DADOS NA PROGRAMAÇÃO

Dados são informações que podem ser armazenadas e manipuladas por um programa.

PRIMITIVOS (SIMPLES)

INTEIRO (int)

Números inteiros, positivos ou negativos, sem casas decimais.

123

Exemplos:

-10, 0, 25, 9876

Em Python:

```
x = 42
```

PONTO FLUTUANTE (float)

Números reais, com parte decimal.

3.14

Exemplos:

-2.5, 0.0, 3.1415, 8.75

Em Python:

```
pi = 3.1415
```

CARACTERE (char)

Representa um único caractere ou símbolo.

'A'

Exemplos:

'A', 'b', '7', '@'

Em Python:

```
letra = 'A'
```

BOOLEANO (bool)

Representa valores lógicos: verdadeiro ou falso.

true
false

Exemplos:

true, false

Em Python:

```
ativo = True
```

LOGICO (bool)

Assim como booleano, usado para lógica verdadeira ou falsa.



Exemplos:

VERDADEIRO, FALSO

Em Linguagens como C:

```
int flag = 1;
```


TIPOS ABSTRATOS DE DADOS (TAD)

Um TAD define **o que** os dados representam e **quais operações** podem ser realizadas, sem se preocupar com **como** será implementado.

O QUE É UM TAD?

Um Tipo Abstrato de Dado (TAD) é um modelo matemático que descreve um conjunto de valores e um conjunto de operações sobre esses valores.



Especifica o que é feito:

- ✓ Quais valores podem existir
- ✓ Quais operações são permitidas



Não especifica como é feito:

- ✗ Como os dados são armazenados
- ✗ Como as operações são implementadas

POR QUE USAR TAD?



Abstração

Esconde os detalhes de implementação, mostrando apenas o essencial ao usuário.



Modularidade

Permite mudar a implementação sem afetar os programas que usam o TAD.



Reutilização

TADs bem definidos podem ser reutilizados em diferentes projetos.



Confiabilidade

Facilita testes e manutenção do código.

VISÃO GERAL



USUÁRIO

usa as operações
do TAD

TAD
(especificação)

implementado por

IMPLEMENTAÇÃO
(estrutura de dados
e algoritmos)



DADOS

EXEMPLOS DE TADs COMUNS

PILHA (STACK)

Coleção de elementos com acesso pelo topo (LIFO: Last In, First Out).

Operações principais:

- criar()
- empilhar(x)
- desempilhar()
- topo()
- vazia()



Ex.: desfazer ações,
chamadas de função.

FILA (QUEUE)

Coleção de elementos com remoção pelo início (FIFO: First In, First Out).

Operações principais:

- criar()
- enfileirar(x)
- desenfileirar()
- frente()
- vazia()



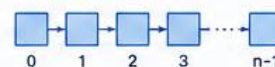
Ex.: filas de atendimento,
impressão de documentos.

LISTA (LIST)

Coleção linear de elementos em que cada posição pode ser acessada.

Operações principais:

- criar()
- inserir(pos, x)
- remover(pos)
- obter(pos)
- tamanho()



Ex.: listas de contatos,
playlists, documentos.

CONJUNTO (SET)

Coleção de elementos únicos, sem ordem.

Operações principais:

- criar()
- inserir(x)
- remover(x)
- pertence(x)
- uniao(outroConjunto)
- intersecao(outroConjunto)



Ex.: eliminar duplicatas,
verificar pertencimento.

DICIONÁRIO (MAP)

Coleção de pares chave-valor (associação).

Operações principais:

- criar()
- inserir(chave, valor)
- remover(chave)
- obter(chave)
- contem(chave)

chave	valor
"nome"	"Ana"
"idade"	22
...	...

Ex.: índices, cadastros,
configurações.



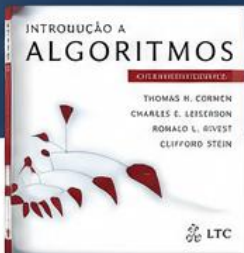
IDEIA-CHAVE:

Um TAD define o contrato entre o usuário e a estrutura de dados. O usuário usa as operações sem precisar saber os detalhes internos de armazenamento e implementação.



BENEFÍCIO:

Maior clareza, segurança e flexibilidade no desenvolvimento de programas.



TEMPO DE EXECUÇÃO DE UM ALGORITMO

Conceito de acordo com Cormen, Leiserson, Rivest e Stein – Introdução a Algoritmos (CLRS)



DEFINIÇÃO (CLRS)

O tempo de execução de um algoritmo é a quantidade de operações elementares executadas como função do tamanho da entrada.

Denotamos por $T_A(n)$ o tempo de execução de um algoritmo A em uma entrada de tamanho n .

Exemplo:

Se um laço executa n vezes e o corpo do laço realiza uma operação constante, então $T_A(n) = cn$ para alguma constante c .



IDEIA PRINCIPAL (CLRS)

Estamos interessados em como o tempo de execução cresce à medida que o tamanho da entrada n aumenta.

Ignoramos constantes e termos de ordem inferior, focando na taxa de crescimento assintótica.



1. OPERAÇÕES ELEMENTARES

O tempo é medido contando o número de operações elementares executadas. As operações elementares dependem da máquina e da linguagem, mas típicas incluem:

- Aritméticas: $+$, $-$, $\times$, $\div$
- Atribuições
- Comparações: $<$, $\leq$, $>$, $\geq$, $=$, $\neq$
- Acesso a elementos de vetor/matriz
- Chamada e retorno de procedimento



2. FUNÇÃO DE TEMPO

Para cada entrada de tamanho n , definimos $T_A(n)$ como o número de operações elementares executadas pelo algoritmo A .

EXEMPLO: Soma de n elementos de um vetor

```
1 SOMA-ARRAY(A[1..n])
2   soma ← 0
3   for i ← 1 to n
4     soma ← soma + A[i]
5   return soma
```

Contagem de operações:

linha 2: 1 operação
linha 3: $(n + 1)$ operações
linha 4: n operações
linha 5: 1 operação

Total: $T(n) = 2n + 2 = \Theta(n)$



3. ANÁLISE ASSINTÓTICA

Usamos notações assintóticas para descrever o crescimento de $T_A(n)$ quando n tende ao infinito.

- $O(g(n))$ – limite superior assintótico
- $\Omega(g(n))$ – limite inferior assintótico
- $\Theta(g(n))$ – limite assintótico exato

Definições (CLRS)

$T(n) = O(g(n))$ se existem $c > 0$, $n_0 \geq 1$ tais que $0 \leq T(n) \leq c g(n)$ para todo $n \geq n_0$.

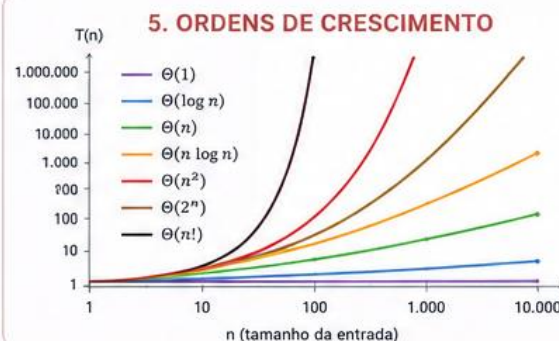
$T(n) = \Omega(g(n))$ se existem $c > 0$, $n_0 \geq 1$ tais que $0 \leq c g(n) \leq T(n)$ para todo $n \geq n_0$.

$T(n) = \Theta(g(n))$ se existem $c_1, c_2 > 0$, $n_0 \geq 1$ tais que $0 \leq c_1 g(n) \leq T(n) \leq c_2 g(n)$ para todo $n \geq n_0$.



4. EXEMPLOS DE ANÁLISE

Algoritmo / Código	$T(n)$	Crescimento
Acesso a elemento de vetor	c	$\Theta(1)$ (constante)
Laço simples (n iterações)	$cn + d$	$\Theta(n)$ (linear)
Dois laços aninhados ($n \times n$)	$cn^2 + dn + e$	$\Theta(n^2)$ (quadrático)
Três laços aninhados (n^3)	$cn^3 + \dots$	$\Theta(n^3)$ (cúbico)
Divide e Conquista (ex.: merge sort)	$cn \log n + dn$	$\Theta(n \log n)$
Todas as permutações	$n!$	$\Theta(n!)$ (fatorial)



Ordem (da menor para a maior)

$\Theta(1) < \Theta(\log n) < \Theta(n)$
 $< \Theta(n \log n) < \Theta(n^2)$
 $< \Theta(2^n) < \Theta(n!)$

Quanto mais à direita na lista, mais rápido o tempo cresce com o aumento de n .



RESUMO (CLRS)

- ✓ O tempo de execução é contado em termos de operações elementares.
- ✓ Expressamos o tempo como uma função $T(n)$ do tamanho da entrada n .
- ✓ Usamos notações assintóticas (O , Ω , Θ) para descrever o crescimento quando n é grande.
- ✓ Focamos na taxa de crescimento, ignorando constantes e termos de ordem inferior.



IMPORTANTE

A análise de tempo de execução nos permite prever o desempenho de algoritmos para entradas grandes e comparar diferentes abordagens para o mesmo problema.

CÁLCULO DO TEMPO DE EXECUÇÃO DE ALGORITMOS

Duas abordagens complementares para avaliar o desempenho de um algoritmo

1. ANÁLISE EMPÍRICA (EXPERIMENTAL)

Mede o tempo executando o algoritmo em uma máquina real.

COMO FUNCIONA



EXEMPLO PRÁTICO

```
import time

def soma_array(A):
    s = 0
    for x in A:
        s += x
    return s

for n in [10**2, 10**3, 10**4, 10**5, 10**6]:
    A = list(range(n))
    inicio = time.perf_counter()
    soma_array(A)
    fim = time.perf_counter()
    print(n, fim - inicio)
```



CARACTERÍSTICAS

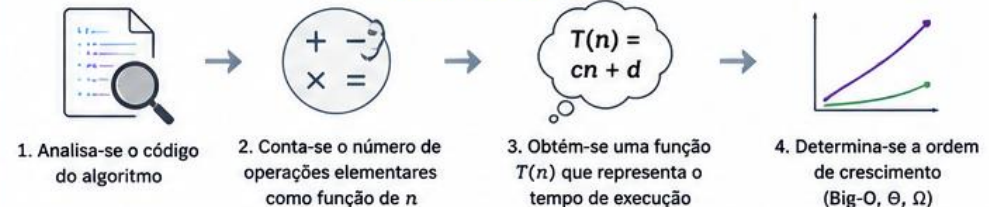
- ✓ Baseada em execução real
- ✓ Depende da máquina, linguagem, compilador, sistema operacional
- ✓ Resultados para entradas específicas
- ✓ Útil para validar desempenho prático e comparar implementações

VS

2. ANÁLISE TEÓRICA (ASSINTÓTICA)

Estima o tempo analisando o algoritmo sem executá-lo.

COMO FUNCIONA



EXEMPLO TEÓRICO

```
def soma_array(A):
    s = 0          # 1 operação
    for x in A:    # n iterações
        s = s + x  # 2 operações por iteração
    return s       # 1 operação
```

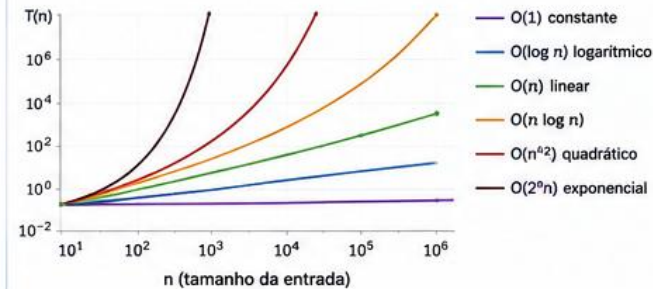
CONTAGEM DE OPERAÇÕES

1 (inicialização) = 1
2 (por iteração) × n = 2n
1 (retorno) = 1

$$T(n) = 2n + 2$$

$$T(n) = \Theta(n)$$

ORDENS DE CRESCIMENTO MAIS COMUNS



CARACTERÍSTICAS

- ✓ Independente de máquina
- ✓ Válida para qualquer entrada de tamanho n
- ✓ Mostra o comportamento de crescimento
- ✓ Permite prever escalabilidade para entradas grandes

EMPÍRICO

- Executa e mede
- Resultado numérico
- Depende do ambiente
- Melhor para entradas reais

COMPARAÇÃO

Execução real	Base	Análise matemática
Máquina, software, hardware	Dependência	Independente
Alta para casos testados	Precisão	Geral (assintótica)
Cronômetros, benchmarks	Ferramentas	Contagem de operações
0,0285 s para $n = 10^6$	Exemplo de resultado	$T(n) = 2n + 2 = \Theta(n)$

TEÓRICO

- Analisa sem executar
- Resultado em função de n
- Independente do ambiente
- Melhor para prever crescimento

CONCLUSÃO

As duas abordagens são complementares: a análise empírica valida na prática e a análise teórica explica o comportamento e permite prever o desempenho para entradas muito grandes.



OBJETIVO FINAL: Escolher algoritmos eficientes e escaláveis para resolver problemas de forma eficaz!

n = tamanho da entrada
 $T(n)$ = tempo de execução para entrada de tamanho n

Conclusão

- Estruturas organizam
- Algoritmos resolvem
- Complexidade otimiza